МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ «КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ імені ІГОРЯ СІКОРСЬКОГО»
Навчально-науковий інститут прикладного системного аналізу Кафедра системного проєктування

КУРСОВА РОБОТА

з дисципліни «Обробка надвеликих масивів даних»

на тему «Побудова асоціативних правил для наборів даних»

Виконав: студент групи КН-51мп
Сидько Нікіта Сергійович
спеціальність 122 «Комп’ютерні науки»
Національна оцінка _____
Кількість балів: _____

Прийняв: проф., д.т.н.
Рогоза Валерій Станіславович
(підпис) (вчене звання, науковий ступінь, ПІБ) 4

Засвідчую, що у цій курсовій роботі
немає запозичень з праць інших авторів
без відповідних посилань
студент Сидько Н.С.

Київ – 2025

Зміст

1. Вступ.....3
2. Теоретичні відомості.....4
2.1. Транзакція та предметний набір.....4
2.2. Асоціативне правило X –> Y.....6
2.3 Кількісні характеристики асоціативних правил.....7
2.3.1. Підтримка асоціативного правила.....8
2.3.2. Вірогідність (достовірність) правила.....8
2.3.3. Цікавість асоціативного правила.....9
2.4. Алгоритм Apriori.....10
3. Хід роботи.....12
3.1 Формулювання задачі курсової роботи.....12
3.2 Формування вхідних даних.....12
3.3 Реалізація алгоритму обчислення метрик асоціативних правил.…13
3.3.1 Функція обчислення підтримки support_itemset.....13
3.3.2 Функція обчислення вірогідності правила confidence.....14
3.3.3 Функція обчислення цікавості правила interest.....15
3.4 Формування асоціативних правил.....15
3.5 Результати роботи коду.....16
3.6 Автоматичне виявлення асоціативних правил методом Apriori.…17
3.6.1 Вхідні дані для Apriori.....18
3.6.2 Реалізація алгоритму Apriori.....18
3.6.3 Генерація асоціативних правил з частих наборів.....19
3.6.4 Результати роботи Apriori.....20
4. Аналіз результатів.....21
4.1 Аналіз результатів для ручних правил.....21
4.2 Аналіз результатів, отриманих за допомогою алгоритму Apriori....22
5.Висновок.....23
6. Список використаних джерел.....24
7. Додаток 1.....25

1. Вступ

В сучасних інформаційних системах постійно накопичуються великі обсяги даних: записи про покупки в супермаркетах, логи відвідувань веб-ресурсів, медичні картки пацієнтів, історія операцій у банківських системах тощо. Самі по собі такі дані не приносять користі, доки з них не виділено приховані закономірності, які допомагають приймати обґрунтовані рішення.

Одним із важливих напрямів аналізу таких даних є побудова асоціативних правил, що дозволяють виявляти стійкі зв’язки між подіями чи об’єктами в транзакціях. Асоціативні правила описують залежності між предметами у вигляді імплікації “якщо в транзакції присутній набір X, то з певною імовірністю в ній буде присутній і набір Y”. Класичним прикладом є аналіз «ринкового кошика» в торгівлі, коли на основі історії покупок визначають, які товари часто купуються разом або, навпаки, майже ніколи не зустрічаються в одній покупці.

Практичне значення асоціативного аналізу полягає в тому, що результати можуть використовуватися для оптимізації викладки товарів у магазинах, побудови рекомендаційних систем (разом із цим товаром часто купують), планування маркетингових акцій, аналізу поведінки користувачів веб-ресурсів, а також для виявлення ризикових або небажаних комбінацій подій у медицині та фінансовій сфері. Навіть на відносно невеликих наборах даних важливо мати формальний апарат, який дозволяє коректно оцінювати асоціативні правила за такими показниками, як підтримка, вірогідність і цікавість. Ці характеристики дозволяють відрізнити випадкові співпадіння від дійсно інформативних закономірностей, що мають практичну цінність.

У даній курсовій роботі розглядається задача побудови асоціативних правил для набору транзакцій, що описують вміст «ринкових кошиків». На основі прикладу з навчального посібника формується множина транзакцій, у кожній з яких зафіксовано набір товарів, придбаних за одну покупку. На цій множині обирається кілька асоціативних правил виду X -> Y з одним або двома предметами в умові та наслідку, для яких обчислюються три основні характеристики: підтримка, що показує, у скількох транзакціях одночасно присутні X і Y; вірогідність, яка відображає частку транзакцій з X, у яких додатково зустрічається Y; а також цікавість, що дозволяє оцінити, наскільки правило відрізняється від випадкового співпадіння.

Метою роботи є розробка та програмна реалізація алгоритму обчислення цих характеристик для заданого набору транзакцій мовою Python, проведення обчислювального експерименту та аналіз інформативності отриманих асоціативних правил.

2. Теоретичні відомості

2.1. Транзакція та предметний набір

Базовим поняттям у теорії асоціативних правил є транзакція. Під транзакцією розуміють деяку множину подій або предметів, що відбуваються спільно. У контексті аналізу «ринкового кошика» транзакція – це набір товарів, які покупець придбав за одну покупку. У більшості реальних ситуацій в супермаркеті клієнт купує не один, а декілька різних товарів, які формують певну комбінацію. Саме ця комбінація і вважається однією транзакцією, тобто одним записом у базі даних для подальшого аналізу.

Загалом усі транзакції, зібрані за деякий період, утворюють базу транзакцій. Формально її можна подати у вигляді множини :

$$D=\{d1, d2, \ldots dN\}, \text{ де}$$

d_i - та транзакція,
N - загальна кількість транзакцій у базі.

Кожна транзакція є підмножиною деякої універсальної множини предметів I, яка містить усі можливі товари, що можуть зустрічатися в кошиках: фрукти, овочі, солодощі тощо. Таким чином, для кожного і виконується співвідношення $d_i \subseteq I$.

У даній роботі в якості прикладу використовується база з десяти транзакцій, наведена в таблиці 1 (взято з навчального посібника), де в кожному рядку перелічено товари, що потрапили до відповідного кошика.

№ транзакції	**Транзакція (кошик)**
1	Сливи, салат, помідори
2	Селера, цукерки
3	Цукерки
4	Яблука, морква, помідори, картопля, цукерки
5	Яблука, апельсини, салат, цукерки, помідори
6	Персики, апельсини, селера, помідори
7	Квасоля, салат, помідори
8	Апельсини, салат, морква, помідори, цукерки
9	Яблука, банани, сливи, морква, помідори, цибуля, цукерки
10	Яблука, картопля

Табл.1 Приклад транзакцій

Важливо підкреслити, що при побудові асоціативних правил нас цікавить лише факт присутності чи відсутності певних товарів у транзакції, а не їх кількість. Тому однаково розглядаються покупки “1 пачка цукерок” і “3 пачки цукерок”: у базі транзакцій вони фіксуються як наявність товару цукерки в кошику. Такий підхід дозволяє зосередитися саме на спільному появленні предметів, що є ключовим для асоціативного аналізу.

Наступним важливим поняттям є предметний набір. Предметним набором називають будь-яку непорожню підмножину множини предметів . Інакше кажучи, це деякий набір товарів, який теоретично може зустрітися у транзакції. Наприклад, множини {салат}, {помідори}, {салат, помідори} або {яблука, цукерки} є предметними наборами, також використовують термін itemset, а для позначення предметних наборів (X Y Z)

Кажуть, що транзакція d_i містить предметний набір X, якщо всі елементи набору X входять доскладу транзакції. Формально це означає, що виконується включення $X \subseteq d_i$. Наприклад, у базі кошиків з таблиці 1 предметний набір {салат, помідори} міститься у чотирьох транзакціях – саме в тих, де одночасно придбані і салат, і помідори. Натомість набір {салат, картопля} може зовсім не зустрічатися в жодній транзакції.

Для зручності подальшого викладу основні позначення, що використовуються в задачі побудови асоціативних правил, зведено в табл. 2 (див. нижче).

Позначення	**Опис**
I	множина всіх предметів (товарів) у базі даних
d_i	і-та транзакція (ринковий кошик)
D	база транзакцій: D = {d_1, d_2, …, d_N}
X	предметний набір (itemset), непорожня підмножина I
$X \subseteq d_i$	транзакція d_i містить предметний набір X

Табл.2 Основні позначення

У подальшому саме предметні набори використовуватимуться для побудови асоціативних правил. Кожне таке правило пов’язує між собою два предметні набори X та Y і описує залежність виду “якщо в транзакції присутній X, то з певною імовірністю присутній і Y”. Формальне означення асоціативного правила X –> Y наведено в підпункті 2.2.

2.2. Асоціативне правило X –> Y

На основі введених понять транзакції та предметного набору формулюється ключове поняття асоціативного аналізу – асоціативне правило. Асоціативне правило описує залежність між двома предметними наборами і має вигляд імплікації

, де

X та Y – предметні набори (непорожні підмножини множини I, причому зазвичай вважається, що вони не перетинаються, тобто . Набір X називають умовою правила, а набір Y – наслідком. Інтуїтивно правило X –> Y читається як “якщо в транзакції присутні всі предмети з набору X, то з певною імовірністю в ній будуть присутні і всі предмети з набору Y”

Важливо розуміти, що таке правило не є строгою логічною імплікацією, яка повинна виконуватись без винятків. У реальних даних завжди існують транзакції, де в кошику є предмети з X, але немає всіх предметів з Y. Тому асоціативне правило розглядається як статистична закономірність, ступінь “сили” та “надійності” якої оцінюється спеціальними показниками, такими як підтримка, вірогідність та цікавість.

У контексті даної роботи множина предметів I відповідає товарам із таблиці 1: “яблука”, “апельсини”, “сливи”, “банани”, “салат”, “помідори”, “картопля”, “морква”, “квасоля”, “селера”, “персики”, “цибуля», “цукерки”. На основі цієї множини можна сформувати велику кількість можливих асоціативних правил, а саме:

1);
;
;
4).

Усі ці правила мають однакову загальну форму, але їхній зміст і «сила» можуть суттєво відрізнятися. Наприклад, правило , означає: “якщо покупець поклав у кошик салат, то, скоріш за все, він купить і помідори”. При аналізі таблиці 1 можна побачити, що всі транзакції, у яких присутній салат (кошики 1, 5, 7, 8), дійсно містять і помідори, тоді це правило в даному наборі даних є досить вагомим.

Розглянемо наступне правило, . На перший погляд схоже, але його зміст інший: “якщо покупець купує помідори, то, можливо, він візьме і салат”. У даних помідори зустрічаються частіше, ніж салат, і є кошики, де помідори присутні без салату (наприклад, транзакції 4, 6, 9). Тому це правило виконується не для всіх транзакцій з помідорами і, як наслідок, має меншу

“силу”, ніж попереднє. Це добре ілюструє, що правило X -> Y загалом не еквівалентне правилу Y –> X, навіть якщо у формулі фігурують ті самі товари.

Щоб зв’язати формальний запис з аналізом конкретних транзакцій, часто розглядають множини індексів транзакцій, у яких виконуються певні умови. Позначимо через T(X) множину номерів транзакцій, що містять предметний набір X. Тоді правило X –>Y “підтримується” тими транзакціями, для яких одночасно присутні всі предмети з X та Y, тобто транзакціями з множини T(X∪Y). Надалі саме на основі множин T(X) та T(X ∪ Y) обчислюються кількісні характеристики асоціативних правил.

№ правила	**Формальний запис**	**Вербальне формулювання**
1	{салат}->{томати}	Якщо в кошику є салат, то, скоріш за все, у ньому будуть і помідори.
2	{помідори}->{салат}	Якщо в кошику є помідори, то іноді до них додають салат.
3	{яблука}->{цукерки}	Покупці, які купують яблука, часто беруть і цукерки.
4	{апельсини}->{томати}	Покупка апельсинів може супроводжуватися покупкою помідорів.

Табл.3 Приклади асоціативних правил для набору транзакцій

2.3 Кількісні характеристики асоціативних правил

У попередніх підпунктах були введені основні об’єкти асоціативного аналізу – транзакції, предметні набори та асоціативні правила вигляду X –> Y. Однак одного лише формального запису правила недостатньо, щоб зробити висновок про його «силу» або практичну корисність. Для цього вводяться спеціальні кількісні характеристики, які обчислюються на основі бази транзакцій. У даній роботі розглядаються три основні метрики:

1. підтримка
2. вірогідність
3. цікавість

2.3.1. Підтримка асоціативного правила

Нехай D = {d_1, d_2 … d_N } це база транзакцій, а X і Y це предметні набори. Підтримка предметного набору X – це кількість транзакцій, в яких присутні всі предмети з набору X. Формально:

;

Для асоціативного правила X –> Y підтримка визначається як підтримка об'єднання $X \cup Y$:

;

Це число показує, у скількох транзакціях одночасно зустрічаються предмети і з умови X, і з наслідку Y. У відносних одиницях підтримку можна нормувати на загальну кількість транзакцій:

;

У контексті прикладу з ринковими кошиками (табл. 1) підтримка правила салат} –> {помідори} дорівнює 4, оскільки в чотирьох транзакціях (1, 5, 7, 8) салат і помідори присутні разом. У відносному вигляді це 4/10=0.4, тобто 40 % усіх покупок.

2.3.2. Вірогідність (достовірність) правила

Друга важлива характеристика – вірогідність (confidence) правила X–>Y. Вона показує, з якою часткою випадків наслідок Y зустрічається в тих транзакціях, де вже присутня умова X. Формально:

;

Тобто ми беремо всі транзакції, що містять X, і дивимося, у якій частині з них одночасно присутній і Y. Значення вірогідності завжди лежить у межах від 0 до 1:

1. значення, близьке до 1, означає, що правило майже завжди виконується, коли трапляється умова X;
2. значення, близьке до 0, свідчить, що наслідок Y (рідко з'являється разом з умовою X).

Для прикладу (з таблички 1) маємо:

1. Правило {салат} –> {помідори}: салат зустрічається у 4 транзакціях, і в усіх цих 4 транзакціях є помідори, отже

2. правило {помідори} –>{салат}: помідори зустрічаються у 7 транзакціях, але салат разом з ними лише в 3 (№ 5, 7, 8), тому

Обидва правила мають однакову підтримку для об’єднання {салат, помідори} (4 транзакції), але їх вірогідність суттєво різниться: у першому випадку вона дорівнює 1, а в другому – менше половини. Це підкреслює важливий факт: правило X –> Y не є еквівалентним правилу Y –>X, порядок набора предметів має значення.

2.3.3. Цікавість асоціативного правила

Третя метрика це цікавість (interest), вона дозволяє оцінити, наскільки правило X –> Y відрізняється від випадкового співпадіння. Ідея полягає в тому, щоб порівняти вірогідність правила з загальною часткою транзакцій, що містять наслідок Y, незалежно від умови, вона визначається так:

,

де , це частка транзакцій, у яких предметний набір Y зустрічається взагалі.

Можливі ситуації :

1. якщо interest ≈ 0 то наявність X майже не змінює ймовірність появи Y
2. якщо interest > 0, то наслідок Y з’являється частіше, коли присутній X ніж у середньому по всій базі транзакцій;
3. якщо interest < 0 то наявність X зменшує ймовірність появи Y можлива несумісність предметів).

Розглянемо знову приклад з помідорами і салатом. Салат зустрічається у 4 транзакціях з 10, отже маємо таке:

Для правила {помідори} -> {салат} ми вже знайшли confidence = 0.43, тоді маємо наступне:

Отримане значення є додатним, але дуже малим і близьким до нуля. Це означає, що наявність помідорів у кошику лише трохи збільшує ймовірність появи салату порівняно із середньою частотою салату в усіх транзакціях, тобто така асоціація не є особливо сильною або несподіваною.

Натомість для правила {салат} -> {помідори} вірогідність дорівнює 1, а частка транзакцій з помідорами становить 0.7 , тоді маємо такі обрахунки:

Таким чином підтримка, вірогідність і цікавість разом дають змогу комплексно оцінювати асоціативні правила. Підтримка відображає, наскільки правило представлене у базі, вірогідність відобразить наскільки воно надійне при появі умови, а цікавість – чи відрізняється виявлена залежність від випадкової співпадності.

2.4. Алгоритм Apriori

Алгоритм Apriori є одним із класичних методів пошуку частих предметних наборів та побудови асоціативних правил у транзакційних базах даних. Основна ідея алгоритму полягає в тому, що якщо певний набір предметів зустрічається недостатньо часто, то будь-яка його надмножина також не може бути частою. Така властивість називається принципом спадання або downward-closure property й дозволяє значно скоротити кількість комбінацій, які необхідно перевіряти в процесі аналізу.

Робота алгоритму є ітеративною і починається з виявлення частих одноелементних наборів. На першому кроці алгоритм обчислює підтримку кожного предмета в усій базі транзакцій та формує множину L1, що складається з тих предметів, чия підтримка не менша за встановлений поріг мінімальної підтримки. Надалі кожна наступна ітерація базується на попередній: з множини частих наборів (Lk−1) генеруються кандидати (Ck) - потенційні часті k-елементні набори, які утворюються шляхом об’єднання частих наборів попереднього рівня. На цьому етапі алгоритм також відкидає всі ті кандидати, чиї підмножини не є частими, оскільки вони завідомо не можуть задовольняти поріг підтримки.

Після формування множини кандидатів Ck для кожного набору обчислюється його підтримка. Набори, що задовольняють умову підтримки, формують множину Lk. Процес повторюється доти, доки множина частих наборів чергового рівня не стане порожньою. У результаті алгоритм генерує об’єднання всіх частих наборів різних рівнів L1, L2, ..., Lk, що можуть бути використані для подальшого формування асоціативних правил.

На заключному етапі Apriori будує асоціативні правила виду X –> Y на основі знайдених частих наборів. Для кожного набору розглядаються всі можливі поділи на умову X та наслідок Y, після чого обчислюється вірогідність правила. Якщо вірогідність перевищує встановлений поріг, правило включається до результатів. Таким чином, Apriori забезпечує систематичний механізм переходу від частих наборів до корисних асоціативних правил.

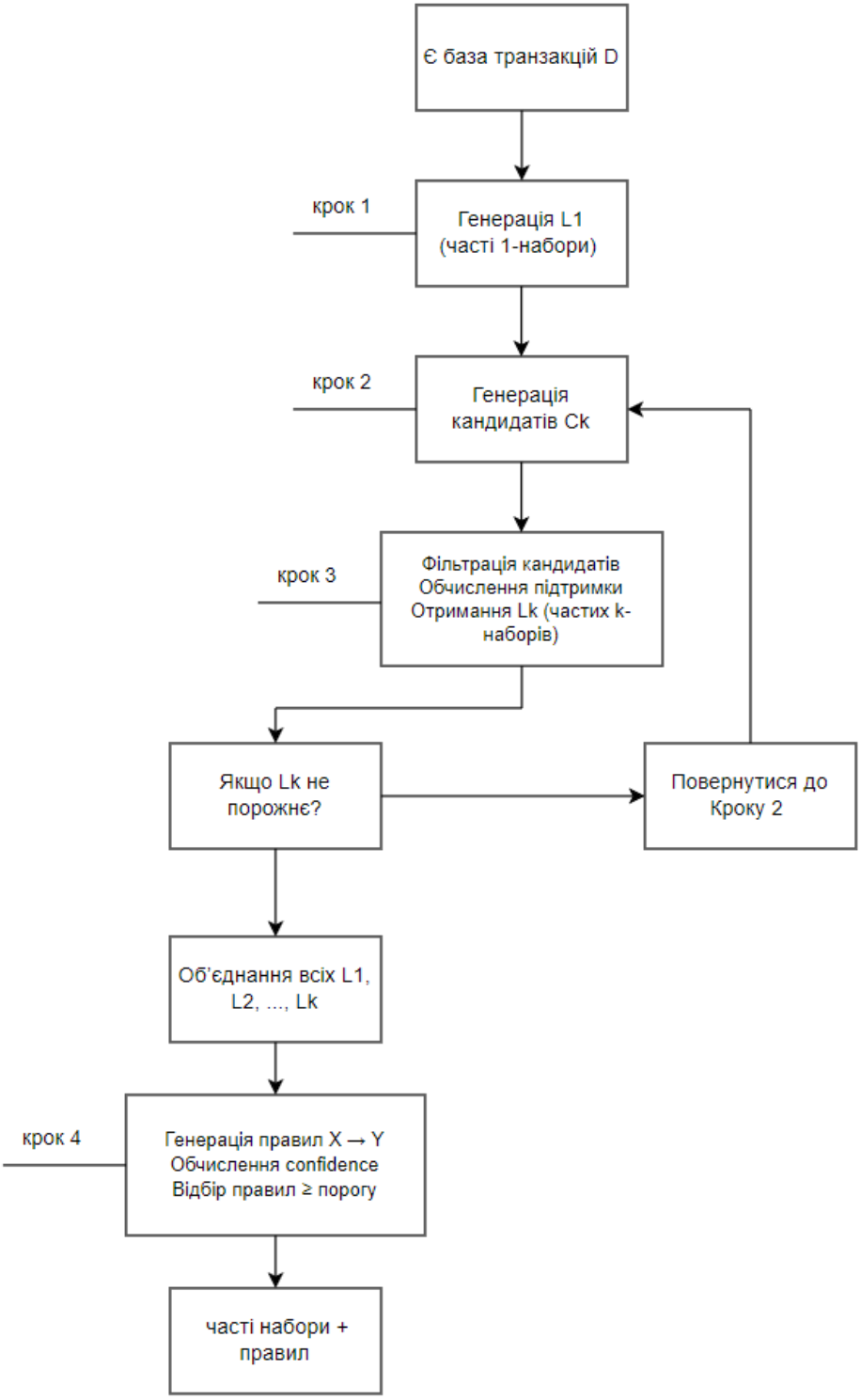


Рис.1 Схема алгоритму Apriori

3. Хід роботи

3.1 Формулювання задачі курсової роботи

Метою роботи є розробка та програмна реалізація алгоритму побудови асоціативних правил для заданого набору транзакцій, а також обчислення і

порівняння характеристик отриманих правил на основі кількісних метрик підтримки, вірогідності та цікавості

Формулювання задачі : є множина транзакцій, кожна транзакція представляє собою набір товарів, придбаних під час однієї покупки. Необхідно реалізувати алгоритм, який для заданих асоціативних правил X –> Y обчислює такі характеристики:

1. підтримку правила - кількість транзакцій, де одночасно зустрічаються предмети з множини X та множини Y
2. вірогідність - частку транзакцій, що містять X, і також присутній Y
3. цікавість - відхилення вірогідності правила від частоти появи Y в усіх транзакціях

Потрібно обрати 5 – 6 асоціативних правил, у яких умова X та наслідок Y містять один або два предмети. Для заданої множини транзакцій обчислити три характеристики для кожного правила, подати результати у табличному вигляді та виконати їх аналіз.

Асоціативні правила вибираються самостійно, допускається використання правил (1 - 1) або (2 - 1, 1 - 2, 2 - 2).

Умови задачі наступні: використання множини транзакцій, наведених у таблиці 1, необхідно побудувати 5 - 6 асоціативних правил, у яких умова X та наслідок Y складаються з одного або двох предметів. Для кожного правила потрібно обчислити три основні характеристики:

1. підтримку,
2. вірогідність (confidence)
3. цікавість (interest)

Реалізація буде виконана на мові Python, а всі отримані результати повинні бути зведені у таблицю для подальшого аналізу.

3.2 Формування вхідних даних

Вхідними даними для побудови асоціативних правил є множина транзакцій, яка відображає структуру покупок у вигляді наборів товарів, що одночасно зустрічаються у ринкових кошиках. У роботі використано транзакції з таблиці 1. Транзакції містять десять покупок, у яких присутні різні комбінації фруктів, овочів та інших продуктів. Кожна транзакція представляється як непорожня множина предметів, придбаних в межах однієї операції. Для подальшої програмної обробки транзакції подано у вигляді списку множин (див. рис. 2)

```
DEFAULT_TRANSACTIONS = [
    {"сливи", "салат", "помідори"},
    {"селера", "цукерки"},
    {"цукерки"},
    {"яблука", "морква", "помідори", "картопля", "цукерки"},
    {"яблука", "апельсини", "салат", "цукерки", "помідори"},
    {"персики", "апельсини", "селера", "помідори"},
    {"квасоля", "салат", "помідори"},
    {"апельсини", "салат", "морква", "помідори", "цукерки"},
    {"яблука", "банани", "сливи", "морква", "помідори", "цибуля", "цукерки"},
    {"яблука", "картопля"}
]
```

Рис. 2 Транзакції

Кожна множина містить унікальні назви товарів. Такий формат зручний для виконання операцій перевірки входження предметних наборів та дозволяє коректно застосовувати операції над множинами під час розрахунку характеристик асоціативних правил.

3.3 Реалізація алгоритму обчислення метрик асоціативних правил

Для реалізації побудови асоціативних правил у програмі необхідно вміти обчислювати три основні характеристики: підтримку предметних наборів, вірогідність правила та його цікавість. У даній роботі ці метрики реалізовані у вигляді трьох окремих функцій на мові Python: support_itemset, confidence та interest. Кожна з них приймає на вхід множини предметів X і Y, а також список транзакцій, та повертає відповідну числову оцінку. Нижче детально розглянуто роботу кожної функції.

3.3.1 Функція обчислення підтримки support_itemset

Функція support_itemset обчислює підтримку предметного набору X, тобто кількість транзакцій, у яких одночасно присутні всі предмети з цього набору (підсумовує кількість транзакцій, які містять усі предмети з X). Нижче наведена частина коду з фунцією (див.рис.3)

```
def support_itemset(X, transactions):  4 usages
    return sum(1 for t in transactions if X.issubset(t))
```

Рис. 3 Функція support_itemset

1. Аргумент X – це множина товарів, наприклад { “салат”, “помідори” }.
2. Аргумент transactions – список усіх транзакцій, де кожна транзакція також задана у вигляді множини товарів.

Всередині функції є вираз: sum(1 for t in transactions if X.issubset (t)). Він перебирає всі транзакції t у списку transactions і для кожної транзакції перевіряє умову X.issubset(t). Виклик методу issubset означає: чи є множина X підмножиною транзакції t. Якщо X входить повністю до складу t, умова істинна, і для цієї транзакції в суму додається одиниця. Якщо хоча б один з елементів X у транзакції відсутній, умова не виконується, і для даної транзакції нічого не додається.

3.3.2 Функція обчислення вірогідності правила confidence

Функція confidence обчислює вірогідність (достовірність) асоціативного правила X –> Y. Іншими словами, вона показує, у якій частці транзакцій, що містять X, додатково присутні й усі предмети з Y. Приклад функції наведений нижче (див.рис.4)

```
def confidence(X, Y, transactions):  2 usages
    sup_X = support_itemset(X, transactions)
    if sup_X == 0:
        return 0.0
    sup_XY = support_itemset(X.union(Y), transactions)
    return sup_XY / sup_X
```

Рис.4 Функція confidence

На першому кроці обчислюється підтримка тільки умови X: sup_X = support_itemset(X, transactions). Тут використовується попередньо описана функція support_itemset. Значення sup_X дорівнює кількості транзакцій, в яких зустрічається набір X. Якщо в базі немає жодної транзакції з X, тобто якщо sup_X == 0, то правило X –> Y не має сенсу, оскільки немає прикладів, де це X в загалі спостерігається. Щоб уникнути ділення на нуль, у такому випадку функція одразу повертає значення 0.0

Далі потрібно обчислити підтримку об’єднаного набору X ∪ Y, тобто кількість транзакцій, де одночасно присутні всі предмети і з X, і з Y, sup_XY = support_itemset(X.union(Y), transactions). Виклик X.union(Y) створює нову множину, яка містить усі предмети з X та Y без повторень. Потім знову використовується функція support_itemset для підрахунку кількості транзакцій, що містять цей об’єднаний набір. Остаточне значення вірогідності правила обчислюється як відношення: return sup_XY / sup_X

3.3.3 Функція обчислення цікавості правила interest

Функція interest реалізує показник interest асоціативного правила X –> Y. Ця метрика порівнює вірогідність правила з фоновою частотою появи Y у всій базі транзакцій та дозволяє оцінити, чи дійсно X впливає на появу Y, чи спільна поява цих предметів є майже випадковою (див.рис.5)

```
def interest(X, Y, transactions):  1 usage
    N = len(transactions)
    conf = confidence(X, Y, transactions)
    sup_Y = support_itemset(Y, transactions)
    return conf - sup_Y / N
```

Рис.5 Функція interest

На початку визначається загальна кількість транзакцій, Далі за допомогою функції confidence обчислюється вірогідність правила X –> Y. Це значення показує, у якій частці транзакцій з X одночасно зустрічається і Y. Наступний крок це оцінкаа наскільки часто предметний набір Y зустрічається у всіх транзакціях незалежно від X. Для цього обчислюється підтримка Y: sup_Y = support_itemset(Y, transactions).

Поділивши sup_Y на загальну кількість транзакцій N, отримуємо оцінку ймовірності появи Y у довільно вибраній транзакції, а показник цікавості повертаємо як різницю.

Якщо ця різниця додатна, правило можна вважати цікавим: Y з’являється в транзакціях з X частіше, ніж загалом у базі. Якщо значення близьке до нуля, поява X практично не впливає на Y. Від’ємне значення означає, що за наявності X предмети з Y зустрічаються навіть рідше, ніж у середньому, і таке правило має низьку практичну цінність.

3.4 Формування асоціативних правил

Згідно поставленого завдання, необхідно сформувати 5 - 6 асоціативних правил, для яких проводиться подальший розрахунок характеристик підтримки, вірогідності та цікавості. Правила обираються відповідно до вимог завдання: умова X та наслідок Y можуть містити один або два предмети. Це дозволяє дослідити різні типи залежностей між товарами та оцінити силу взаємозв’язків як у простих (1 - 1), так і у комбінованих правилах (2 - 1, 1 - 2, 2 - 2).

Для аналізу було складено набір правил, що базуються на змістовних спостереженнях щодо предметів у вихідних транзакціях. Наприклад, у кошиках досить часто разом зустрічаються салат і помідори, яблука та цукерки, апельсини та овочеві товари. Такі закономірності природно лягають в основу формування правил X –> Y. Нижче наведено приклад коду з правилами (див.рис.6)

```
def use_default_rules():  1 usage
    return [
        ({"салат"}, {"помідори"}),
        ({"помідори"}, {"салат"}),
        ({"яблука"}, {"цукерки"}),
        ({"апельсини"}, {"помідори"}),
        ({"морква"}, {"цукерки"}),
        ({"селера"}, {"апельсини"}),
        ({"яблука", "морква"}, {"цукерки"}),
        ({"апельсини"}, {"помідори", "салат"}),
        ({"сливи", "яблука"}, {"морква", "помідори"})
    ]
```

Рис.6 Асоціативні правила

Наявність правила кожного типу дозволяє оцінити, як змінюються характеристики метрик при ускладненні структури предметних наборів. Наприклад, підтримка та впевненість для наборів з двох елементів можуть бути нижчими, оскільки одночасна поява більшої кількості предметів є менш імовірною.

3.5 Результати роботи коду

Після формування множини асоціативних правил та обчислення для них необхідних метрик програма формує підсумкову таблицю, у якій для кожного правила наведено значення трьох основних характеристик: підтримки (support), вірогідності (confidence) та цікавості (interest). Табличний формат дозволяє наочно порівняти правила між собою та провести змістовний аналіз отриманих залежностей між предметами у транзакціях.

У ході виконання програми результати виводяться у консолі у вигляді форматованої таблиці. Фрагмент коду, що відповідає за формування цього виводу, наведено нижче (див.рис.7):

```
def print_table(results):  2 usages
    col_rule = max(len(r[0]) for r in results) + 3

    print("\nРезультати аналізу асоціативних правил\n")
    print(f"{'Правило'.ljust(col_rule)}Підтримка   Вірогідність   Цікавість")

    for rule, sup, conf, intr in results:
        print(f"{rule.ljust(col_rule)}"f"{str(sup).ljust(12)}"f"{str(conf).ljust(15)}"f"{intr}")
```

Рис.7 Функція print_table

Пояснення показників:

1. Підтримка: визначає кількість транзакцій, у яких одночасно зустрічаються всі предмети з множини X ∪ Y. Висока підтримка свідчить, що правило має широке охоплення і базується на частих комбінаціях товарів.
2. Вірогідність: умовна ймовірність появи множини Y у транзакції за умови, що в ній присутня множина X. Значення наближення до 1 свідчить про сильний напрямлений зв’язок.
3. Цікавість: дозволяє оцінити, наскільки взаємопоява предметів у правилі відрізняється від їхнього випадкового співпадіння. Додатні значення вказують на значущу асоціацію, близькі до нуля - на випадковість, від’ємні - на антагоністичний зв’язок.

Нижче наведено результат роботи коду (див.рис.8)

```
Транзакції:
1. помідори, салат, сливи
2. селера, цукерки
3. цукерки
4. картопля, морква, помідори, цукерки, яблука
5. апельсини, помідори, салат, цукерки, яблука
6. апельсини, персики, помідори, селера
7. квасоля, помідори, салат
8. апельсини, морква, помідори, салат, цукерки
9. банани, морква, помідори, сливи, цибуля, цукерки, яблука
10. картопля, яблука

Результати аналізу асоціативних правил

Правило                          Підтримка   Вірогідність   Цікавість
салат - помідори                 4           1.0            0.3
помідори - салат                 4           0.571          0.171
яблука - цукерки                 3           0.75           0.15
апельсини - помідори             3           1.0            0.3
морква - цукерки                 3           1.0            0.4
селера - апельсини               1           0.5            0.2
яблука,морква - цукерки          2           1.0            0.4
апельсини - салат,помідори       2           0.667          0.267
яблука,сливи - помідори,морква   1           1.0            0.7

Process finished with exit code 0
```

Рис.8 Результат роботи коду

3.6 Автоматичне виявлення асоціативних правил методом Apriori

Окрім основного алгоритму, у рамках даної курсової роботи було проведено додатковий експеримент із застосуванням алгоритму Apriori класичного методу пошуку частих предметних наборів (frequent itemsets) та автоматичної генерації асоціативних правил. Його використання дозволяє не лише підтвердити вручну задані правила, але й виявити нові закономірності, що не були очевидними під час початкового аналізу.

3.6.1 Вхідні дані для Apriori

Для демонстрації роботи алгоритму Apriori використано спрощений навчальний набір транзакцій наведений нижче (див. рис. 9)

```
APRIORI_TRANSACTIONS = [
    {"хліб", "молоко", "печиво"},
    {"хліб", "молоко"},
    {"хліб", "печиво"},
    {"молоко", "печиво"},
    {"хліб", "молоко", "масло"},
    {"хліб", "печиво", "масло"}
]
```

Рис.9 Набір транзакцій для Apriori

3.6.2 Реалізація алгоритму Apriori

Реалізація алгоритму Apriori у програмному коді виконує поетапне виявлення частих предметних наборів у множині транзакцій. Функція автоматично формує часті набори L1, L2, L3 поступово збільшуючи їх розмір, а також застосовує фільтрацію кандидатів на основі мінімальної підтримки. Робота алгоритму складається з побудови початкових одноелементних частих наборів, генерації нових кандидатів, оцінювання їх підтримки та додавання до списку частих наборів лише тих, що відповідають пороговим значенням. Нижче наведено фрагмент програмної реалізації (див.рис.10)

```
def apriori(transactions, min_support):  1 usage
    items = sorted(set(chain.from_iterable(transactions)))
    L1 = []
    for item in items:
        if support_itemset({item}, transactions) >= min_support:
            L1.append(frozenset([item]))

    L = [L1]
    k = 2

    while True:
        prev_L = L[-1]
        candidates = []
        for a, b in combinations(prev_L,  r: 2):
            union = a.union(b)
            if len(union) == k and union not in candidates:
                candidates.append(union)

        Lk = []
        for c in candidates:
            if support_itemset(c, transactions) >= min_support:
                Lk.append(c)

        if not Lk:
            break

        L.append(Lk)
        k += 1

    return list(chain.from_iterable(L))
```

Рис.10 Реалізація алгоритму Apriori

Тут, items це список усіх унікальних товарів у транзакціях. Далі для кожного товару рахується підтримка. Якщо товар зустрічається у транзакціях не менше min_support, він потрапляє до множини частих одноелементних наборів L1 Змінна L зберігає всі знайдені рівні L_k, а k = 2 означає, що наступним кроком буде побудова частих пар (L_2).

Генерація кандидатів виконується у циклі While True, тут prev_L містить часті набори попереднього рівня L_{k-1}, combinations(prev_L, 2) перебирає всі пари частих наборів. Якщо їх об'єднання має рівно k елементів то воно є кандидатом. Таким чином, алгоритм генерує можливі набори розміру k у свою чергу множина кандидатів C_k ще не є частою, бо потребує фільтрації яка виконується у циклі for c in candidates. Для кожного кандидата c обчислюється підтримка. Якщо кандидат зустрічається достатню кількість разі то він потрапляє у часті набори L_k. Якщо L_k = порожній множині, то алгоритм зупиняється. У іншому разі L_k додається до загального списку рівнів, а алгоритм переходить до побудови наборів більшого розміру.

Усі знайдені часті набори з'єднуються в один список, а функція повертає повний набір частих предметних комбінацій.

3.6.3 Генерація асоціативних правил з частих наборів

Після побудови множини частих предметних наборів наступним кроком є автоматична генерація асоціативних правил. Алгоритм Apriori створює усі можливі правила виду X -> Y, де X та Y - непорожні підмножини частого набору, а також обчислює їхню вірогідність (confidence). Якщо значення вірогідності перевищує встановлений поріг, правило вважається значущим і включається до результатів. На рисунку нижче (див.рис.11) подано код функції, що реалізує цей етап.

```python
def generate_rules(frequent_itemsets, transactions, min_confidence):  1 usage
    rules = []

    for itemset in frequent_itemsets:
        if len(itemset) < 2:
            continue

        items = list(itemset)

        for r in range(1, len(items)):
            for X in combinations(items, r):
                X = set(X)
                Y = set(itemset) - X

                conf = confidence(X, Y, transactions)

                if conf >= min_confidence:
                    rules.append((X, Y, round(conf, 3)))

    return rules
```

Рис. 11 Функція generate_rules

Спочатку йде цикл for itemset in frequent_itemsets, Перебираються всі часті набори, знайдені алгоритмом Apriori. Якщо набір містить менше двох елементів - правило побудувати неможливо, тому такий набір пропускається. Далі йде початковий частий набір перетворюється у список items. Після чого у циклі for r in range(1, len(items)) йде генерація всіх можливих підмножин X. Потім йде conf = confidence(X, Y, transactions), воно виконує обчислення вірогідності правила та перевірка порогу.

3.6.4 Результати роботи Apriori

На рисунку нижче зображено результати роботи додаткової частини коду (включно з алгоритмом Apriori), застосованого до спрощеної множини транзакцій, що містить шість покупок із товарами “хліб”, “молоко”, “печиво” та “масло”. На основі цих даних алгоритм виконав побудову множин частих предметних наборів за заданим порогом підтримки, а також згенерував асоціативні правила, які відображають найімовірніші залежності між товарами. Зліва наведено вихідні транзакції та знайдені часті набори, а справа подано таблицю правил з розрахованими метриками підтримки, вірогідності та цікавості.

```
Спрощені транзакції для Apriori:
1. молоко, печиво, хліб
2. молоко, хліб
3. печиво, хліб
4. молоко, печиво
5. масло, молоко, хліб
6. масло, печиво, хліб

Часті набори предметів (Apriori):
масло
молоко
печиво
хліб
хліб, масло
печиво, молоко
хліб, молоко
хліб, печиво
```

```
Таблиця правил Apriori (min_confidence=0.3):

Результати аналізу асоціативних правил

Правило           Підтримка   Вірогідність   Цікавість
хліб - масло      2           0.4            0.067
масло - хліб      2           1.0            0.167
печиво - молоко   2           0.5            -0.167
молоко - печиво   2           0.5            -0.167
хліб - молоко     3           0.6            -0.067
молоко - хліб     3           0.75           -0.083
хліб - печиво     3           0.6            -0.067
печиво - хліб     3           0.75           -0.083

Process finished with exit code 0
```

Рис.12 Результати роботи коду Apriori частини

4. Аналіз результатів

У процесі виконання роботи було отримано набір асоціативних правил як шляхом ручного формування (згідно з завданням теми), так і автоматично за допомогою алгоритму Apriori. Для кожного правила обчислено три основні характеристики: підтримку, вірогідність та цікавість.

4.1 Аналіз результатів для ручних правил

Ручні правила були сформовані на основі множини транзакцій, наведених у таблиці 1. Вони охоплюють прості залежності між товарами, де X та Y складаються з одного або двох предметів. Обчислені характеристики дозволяють охарактеризувати силу зв’язку між товарами, що одночасно з’являються у ринковому кошику.

Одним з найсильніших правил виявилося {салат} -> {помідори}, яке має вірогідність 1,0. Це означає, що в усіх транзакціях, де покупець обирав салат, обов’язково присутні і помідори. Така закономірність може мати практичне застосування у побудові рекомендацій (разом із салатом часто купують помідори) або плануванні розташування товарів у магазині.

Зворотне правило {помідори} -> {салат} має суттєво нижчу довіру. Помідори зустрічаються частіше, тому не кожна транзакція з помідорами включає салат. Це демонструє ключову властивість асоціативних правил вони не симетричні: X -> Y не еквівалентно Y -> X.

Правила типу (2 –> 1), наприклад {яблука, морква} –> {цукерки}, показали високу підтримку, що свідчить про часту спільну появу таких наборів у даних. Подібні правила можуть сигналізувати про типові «поведінкові патерни» покупців – наприклад, певні групи товарів можна купувати для приготування страв.

Правила типу (1 –> 2) мають нижчу підтримку, адже одночасна поява двох предметів у правій частині набагато менш імовірна. Це природно, оскільки складніші правила потребують більшої кількості спільних входжень. Показник цікавості виявився як додатним, так і від’ємним:

1. Додатна цікавість означає, що спільність X і Y є вищою, ніж очікувалося випадково.
2. Від’ємна – що Y з’являється не частіше, ніж загалом у вибірці, тобто правило не додає корисної інформації.

В цілому, ручний аналіз правил дозволив виявити сильні та слабкі зв’язки у множині транзакцій, а також підкреслив обмеження людського вибору правил деякі закономірності могли залишитися непоміченими без автоматичного аналізу.

4.2 Аналіз результатів, отриманих за допомогою алгоритму Apriori

Додатковий експеримент із алгоритмом Apriori було виконано на спрощеній множині транзакцій. Метою було продемонструвати роботу автоматичного механізму виявлення частих наборів предметів та побудови правил без участі користувача.

Apriori автоматично згенерував усі часті предметні набори із заданим порогом підтримки. Серед них виявилися як прості одноелементні набори (наприклад, “молоко”, “печиво”, “хліб”), так і пари товарів, що часто купуються разом.

Подальша генерація асоціативних правил показала, що деякі з них мають дуже високу силу зв’язку. Наприклад, правило {масло} –> {хліб} має довіру 1.0, що означає: у кожній транзакції з маслом присутній і хліб. Такі правила напряму демонструють корисні маркетингові залежності.

Разом з тим, алгоритм згенерував правила з різною мірою довіри – від високої до середньої. Наприклад, правила {печиво} –> {молоко} та {хліб} –> {печиво} мають середній рівень впевненості, але їх поява є статистично обґрунтованою. Показник цікавість деяких правил набув від’ємних значень, що свідчить про слабший зв’язок, ніж можна було очікувати за загальною частотою появи предметів.

На відміну від ручного підходу, Apriori знаходить усі закономірності, які відповідають заданим порогам підтримки та довіри. Це дозволяє виявити приховані залежності, які важко передбачити вручну, і підвищує якість аналітичних висновків.

5.Висновок

У ході виконання курсової роботи було досліджено процес побудови асоціативних правил для наборів транзакцій та проведено аналіз їхніх кількісних характеристик. На основі теоретичних відомостей сформовано розуміння ключових понять асоціативного аналізу, таких як транзакція, предметний набір, підтримка, вірогідність та цікавість правила. Реалізовано програмний модуль, що дозволяє обчислювати зазначені метрики для заданих асоціативних правил та виконувати аналіз залежностей між товарами, які одночасно зустрічаються у покупках.

На основі транзакцій з умови завдання було сформовано 5 - 6 правил, для яких проведено розрахунки усіх необхідних характеристик. Аналіз показав, що окремі правила демонструють сильні та стійкі залежності (наприклад, повну вірогідність), тоді як інші мають слабший або випадковий характер зв’язку. Значення показника цікавості дозволило визначити, наскільки отримані залежності є інформативними, а не лише статистично частими.

Для порівняння ручного підходу з автоматизованим було реалізовано алгоритм Apriori, який дає змогу автоматично знаходити часті предметні набори та генерувати асоціативні правила на їх основі. На спрощених транзакційних даних Apriori продемонстрував здатність виявляти закономірності, що не завжди очевидні при ручному формуванні правил. Це підкреслило важливість методів автоматичного пошуку закономірностей у великих масивах даних.

У результаті виконання роботи було підтверджено, що асоціативні правила є ефективним інструментом для аналізу транзакційних даних, а їх кількісні характеристики дають можливість оцінювати силу та значущість виявлених залежностей. Практичне застосування таких методів може бути корисним у сферах роздрібної торгівлі, маркетингової аналітики, рекомендаційних систем та інших галузях, що потребують аналізу поведінкових патернів користувачів.

6. Список використаних джерел

1. Han J., Kamber M., Pei J. Data Mining: Concepts and Techniques [Електронний ресурс] // Morgan Kaufmann. – Режим доступу: https://www.sciencedirect.com/book/9780123814791/data-mining

2. Agrawal R., Imieliński T., Swami A. Mining Association Rules Between Sets of Items in Large Databases [Електронний ресурс] // ACM SIGMOD Conference. – Режим доступу: https://dl.acm.org/doi/10.1145/170035.170072

3. Tan P.-N., Steinbach M., Kumar V. Introduction to Data Mining [Електронний ресурс] // Pearson Education. – Режим доступу: https://www-users.cse.umn.edu/~kumar001/dmbook/index.php

4. Witten I., Frank E., Hall M., Pal C. Practical Machine Learning Tools and Techniques [Електронний ресурс] // Morgan Kaufmann. – Режим доступу: https://www.elsevier.com/books/practical-machine-learning-tools-and-techniques

5. Raschka S., Mirjalili V. Python Machine Learning: Machine Learning and Deep Learning with Python [Електронний ресурс] // Packt Publishing. – Режим доступу: https://github.com/rasbt/python-machine-learning-book

6. Brin S., Motwani R., Ullman J., Tsur S. Dynamic Itemset Counting and Implication Rules for Market Basket Data [Електронний ресурс] // ACM SIGMOD. – Режим доступу: https://dl.acm.org/doi/10.1145/253260.253325

7. The Python Software Foundation. Документація Python: модуль itertools [Електронний ресурс] // Python Docs. – Режим доступу: https://docs.python.org/3/library/itertools.htmlPandas Development Team.

8. Університетський навчальний посібник «Обробка нечислових множин даних». Тема 12 — Асоціативні правила [Електронний ресурс] // Навчальні матеріали кафедри.

9. GeeksforGeeks. Apriori Algorithm in Data Mining [Електронний ресурс] // GeeksforGeeks. – Режим доступу: https://www.geeksforgeeks.org/apriori-algorithm/

10. StackOverflow Community. Приклади реалізації Apriori на Python [Електронний ресурс] // StackOverflow. – Режим доступу: https://stackoverflow.com/

7. Додаток 1

```
import pandas as pd
from itertools import combinations, chain


DEFAULT_TRANSACTIONS = [
    {"сливи", "салат", "помідори"},
    {"селера", "цукерки"},
    {"цукерки"},
    {"яблука", "морква", "помідори", "картопля", "цукерки"},
    {"яблука", "апельсини", "салат", "цукерки", "помідори"},
    {"персики", "апельсини", "селера", "помідори"},
    {"квасоля", "салат", "помідори"},
    {"апельсини", "салат", "морква", "помідори", "цукерки"},
    {"яблука", "банани", "сливи", "морква", "помідори", "цибуля", "цукерки"},
    {"яблука", "картопля"}
]



APRIORI_TRANSACTIONS = [
    {"хліб", "молоко", "печиво"},
    {"хліб", "молоко"},
    {"хліб", "печиво"},
    {"молоко", "печиво"},
    {"хліб", "молоко", "масло"},
    {"хліб", "печиво", "масло"}
]



def support_itemset(X, transactions):
    return sum(1 for t in transactions if X.issubset(t))


def confidence(X, Y, transactions):
    sup_X = support_itemset(X, transactions)
    if sup_X == 0:
        return 0.0
    sup_XY = support_itemset(X.union(Y), transactions)
    return sup_XY / sup_X


def interest(X, Y, transactions):
    N = len(transactions)
    conf = confidence(X, Y, transactions)
    sup_Y = support_itemset(Y, transactions)
    return conf - sup_Y / N



def use_default_rules():
    return [
        ({"салат"}, {"помідори"}),
        ({"помідори"}, {"салат"}),
        ({"яблука"}, {"цукерки"}),
        ({"апельсини"}, {"помідори"}),
        ({"морква"}, {"цукерки"}),
        ({"селера"}, {"апельсини"}),
        ({"яблука", "морква"}, {"цукерки"}),
```

```
        ({"апельсини"}, {"помідори", "салат"}),
        ({"сливи", "яблука"}, {"морква", "помідори"})
    ]



def apriori(transactions, min_support):
    items = sorted(set(chain.from_iterable(transactions)))
    L1 = []
    for item in items:
        if support_itemset({item}, transactions) >= min_support:
            L1.append(frozenset([item]))

    L = [L1]
    k = 2

    while True:
        prev_L = L[-1]
        candidates = []
        for a, b in combinations(prev_L, 2):
            union = a.union(b)
            if len(union) == k and union not in candidates:
                candidates.append(union)

        Lk = []
        for c in candidates:
            if support_itemset(c, transactions) >= min_support:
                Lk.append(c)

        if not Lk:
            break

        L.append(Lk)
        k += 1

    return list(chain.from_iterable(L))


def generate_rules(frequent_itemsets, transactions, min_confidence):
    rules = []

    for itemset in frequent_itemsets:
        if len(itemset) < 2:
            continue

        items = list(itemset)

        for r in range(1, len(items)):
            for X in combinations(items, r):
                X = set(X)
                Y = set(itemset) - X

                conf = confidence(X, Y, transactions)

                if conf >= min_confidence:
                    rules.append((X, Y, round(conf, 3)))

    return rules



def evaluate_rules(transactions, rules):
    results = []
```

```
    for X, Y in rules:
        rule_str = f"{','.join(X)} - {','.join(Y)}"
        sup = support_itemset(X.union(Y), transactions)
        conf = round(confidence(X, Y, transactions), 3)
        intr = round(interest(X, Y, transactions), 3)
        results.append((rule_str, sup, conf, intr))
    return results


def print_table(results):
    col_rule = max(len(r[0]) for r in results) + 3

    print("\nРезультати аналізу асоціативних правил\n")
    print(f"{'Правило'.ljust(col_rule)}Підтримка   Вірогідність   Цікавість")

    for rule, sup, conf, intr in results:
        print(f"{rule.ljust(col_rule)}"f"{str(sup).ljust(12)}"f"{str(conf).ljust(15)}"f"{intr}")



def main():
    print("Виберіть режим роботи")
    print("1)Основний алгоритм")
    print("2)Додатковий експеримент Apriori на спрощених даних")

    choice = input("Ваш вибір: ")

    if choice == "1":
        transactions = DEFAULT_TRANSACTIONS
        rules = use_default_rules()

        print("\nТранзакції:")
        for i, t in enumerate(transactions, 1):
            print(f"{i}. {', '.join(sorted(t))}")

        results = evaluate_rules(transactions, rules)
        print_table(results)


    elif choice == "2":
        transactions = APRIORI_TRANSACTIONS

        print("\nСпрощені транзакції для Apriori:")
        for i, t in enumerate(transactions, 1):
            print(f"{i}. {', '.join(sorted(t))}")

        frequent = apriori(transactions, min_support=2)

        print("\nЧасті набори предметів (Apriori):")
        for s in frequent:
            print(f"{', '.join(s)}")

        print("\nТаблиця правил Apriori (min_confidence=0.3):")
        raw_rules = generate_rules(frequent, transactions, min_confidence=0.3)


        apriori_rules = [(X, Y) for X, Y, c in raw_rules]

        if not apriori_rules:
            print("Правил не знайдено")
        else:
            apriori_results = evaluate_rules(transactions, apriori_rules)
            print_table(apriori_results)
```

```
    else:
        print("Невірний вибір")


if __name__ == "__main__":
    main()
```